

Homework I:

UNIX System Administration & Security

Module: 5CS035 - Ethical Hacking (Week 1)

Instructions

Complete the following tasks on your Kali Linux environment (VM, WSL, or Labex).

For every task, you must provide a **screenshot** that clearly shows the terminal window, the command executed, and the resulting output. Ensure your username is visible in the prompt.

Task 1: Privilege Escalation & Sudo Configuration

In this task, you will simulate the process of a "Standard User" attempting to gain administrative (Root) access.

1. **Create a New User:** Create a user named lab_user_01.
2. **Initial Restriction Test:** Switch to lab_user_01. Attempt to run a root-only command using sudo.
Note: This should fail with a message stating the user is not in the sudoers file.
3. **Return to Main User:** Exit the lab_user_01 session to return to your main administrative account (or use exit).
4. **Grant Root Privileges:** Add lab_user_01 to the sudo group so they can execute administrative commands.
5. **Final Verification:** Switch back to lab_user_01 and run a command with sudo again. This time, it should succeed.
6. **Required Screenshot:** Show the "Permission Denied" error from step 2 and the successful sudo command from step 5.

ANS:

```
(aadarsh@kali)~$ sudo adduser lab_user_01
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for lab_user_01
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n]

(aadarsh@kali)~$ su - lab_user_01
Password:
(lab_user_01@kali)~$ sudo whoami
[sudo] password for lab_user_01:
lab_user_01 is not in the sudoers file.
```

```
(lab_user_01@kali)-[~]
$ exit
logout

(aadarsh@kali)-[~]
$ sudo usermod -aG sudo lab_user_01

(aadarsh@kali)-[~]
$ su - lab_user_01
Password:
su: Authentication failure

(aadarsh@kali)-[~]
$ su - lab_user_01
Password:
(lab_user_01@kali)-[~]
$ sudo whoami
[sudo] password for lab_user_01:
root

(lab_user_01@kali)-[~]
$ |
```

Task 2: Advanced Identity & Access Management

In this scenario, you will simulate a departmental structure by configuring specific access restrictions and group ownership inheritance:

1. Create a new system group named `security_audit`.
2. Create two new users: `auditor_01` and `dev_02`.
3. Assign `auditor_01` to the `security_audit` group as a **secondary** group.
4. Create a directory located at `/tmp/audit_logs`.
5. Configure the system so that any new file created by `auditor_01` within `/tmp/audit_logs` is automatically owned by the `security_audit` group.

Hints: Use `sudo chgrp security_audit /tmp/audit_logs` to change the directory's group ownership.

- Use `sudo chmod g+s /tmp/audit_logs` to apply the **SetGID** bit, which enforces group inheritance for new files.
-

Submission Requirements

To verify your configuration, provide a single screenshot or text output showing the results of the following commands:

- `id auditor_01` (To verify group membership)
- `ls -l /tmp/audit_logs` (To verify directory permissions and the s bit)

ANS:

```
(aadarsh@kali)~$ sudo groupadd security_audit && sudo useradd auditor_01

(aadarsh@kali)~$ sudo useradd dev_02 && sudo usermod -aG security_audit auditor_01

(aadarsh@kali)~$ sudo mkdir /tmp/audit_logs

(aadarsh@kali)~$ sudo chgrp security_audit /tmp/audit_logs

(aadarsh@kali)~$ sudo chmod g+ws /tmp/audit_logs

(aadarsh@kali)~$

(aadarsh@kali)~$ id auditor_01 && ls -ld /tmp/audit_logs
uid=1004(auditor_01) gid=1005(auditor_01) groups=1005(auditor_01),1004(security_audit)
drwxrwsr-x 2 root security_audit 40 Mar  5 18:02 /tmp/audit_logs

(aadarsh@kali)~$
```

Task 3: Granular Permission Logic

1. Create a file named `top_secret.txt` in your home directory.
2. Using octal notation, configure the file permissions so that:
 - The owner (user) has read and write permissions.
 - The group has read-only permission.
 - Others have no permissions.
3. Modify the file permissions using symbolic or octal notation so that the final permission string appears as:

-rwxrwsrwt

Note: Unix systems support three special permission bits:

- **setuid (SUID)** – Executes the file with the privileges of the file owner.
- **setgid (SGID)** – Executes the file with the privileges of the file group.
- **Sticky bit** – Primarily used on directories to prevent users from deleting files owned by others within the same directory (e.g., /tmp).

4. Create a directory named `shared_drop` and use `chmod` to apply the sticky bit to it.

Note:

The `t` in a directory's permission string indicates that the sticky bit is enabled. It does not prevent execution; it restricts deletion of files by non-owners within that directory. 5. Submit all the required Screenshots

ANS:

```
(aadarsh@kali)-[~]
$ touch top_secret.txt

(aadarsh@kali)-[~]
$ chmod 640 top_secret.txt

(aadarsh@kali)-[~]
$ chmod 3777 top_secret.txt

(aadarsh@kali)-[~]
$ mkdir shared_drop

(aadarsh@kali)-[~]
$ chmod +t shared_drop

(aadarsh@kali)-[~]
$ |
```

```
(aadarsh@kali)-[~]
$ ls -ld shared_drop top_secret.txt
drwxrwxr-t 2 aadarsh aadarsh 4096 Mar  5 18:04 shared_drop
-rwxrwsrwt 1 aadarsh aadarsh   0 Mar  5 18:04 top_secret.txt
```

Task 4: Automated Reconnaissance Scripting

1. Create a Bash script named `sys_recon.sh`.
2. The script must output the following in a clean format, keep it simple, no need to use conditional statements for the following:
 - Current System Date/Time.
 - The Shell currently being used.
 - The last 5 lines of the system authorization log (usually `/var/log/auth.log` or `/var/log/syslog`).
3. Make the script executable only by the owner.
4. **Required Screenshot:** Show the content of the script (`cat sys_recon.sh`) and the successful execution of the script (`./sys_recon.sh`).

ANS:

```
(aadarsh@kali)-[~]
$ sudo ./sys_recon.sh
Current Date/Time:
Thu Mar  5 06:11:00 PM +0545 2026
Current Shell:
/usr/bin/zsh
Last 5 lines of auth log:
Mar 05 18:10:32 kali systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
Mar 05 18:11:00 kali sudo[9007]: aadarsh : TTY=pts/0 ; PWD=/home/aadarsh ; USER=root ; COMMAND=./sys_recon.sh
Mar 05 18:11:00 kali sudo[9007]: pam_unix(sudo:session): session opened for user root(uid=0) by aadarsh(uid=1001)
Mar 05 18:11:00 kali sudo[9012]: root : TTY=pts/1 ; PWD=/home/aadarsh ; USER=root ; COMMAND=/usr/bin/journalctl -n 5
Mar 05 18:11:00 kali sudo[9012]: pam_unix(sudo:session): session opened for user root(uid=0) by aadarsh(uid=0)

(aadarsh@kali)-[~]
$ cat sys_recon.sh
#!/bin/bash
echo "Current Date/Time:"
date
echo "Current Shell:"
echo $SHELL
echo "Last 5 lines of auth log:"
sudo journalctl -n 5

(aadarsh@kali)-[~]
$ |
```

Submission Format

- **Document:** Submit a PDF file.
- Each page should have your name written on the top of the page.
- Submissions should show the terminal window.